

ex 9

User Registration Form with Client-Side Validation

Name:ROHITH R

ROLLNO:240701804

AIM

To design and implement a User Registration Form using HTML, CSS, and JavaScript that performs client-side validation of user inputs — specifically email address and phone number — using the Validator.js library, and provides real-time feedback to the user upon form submission.

PROCEDURE

Step 1: Create the HTML Structure (indexx.html)

1. Create an HTML file with a form container div having a heading "User Registration Form".
2. Add an input group for Email with a text input field (id="email") and an error message placeholder (id="emailError").
3. Add an input group for Phone Number with a text input field (id="phone") and an error message placeholder (id="phoneError").
4. Include a Submit button and a success message paragraph (id="successMessage").
5. Link the Validator.js CDN and the external script file (scriptt.js) before the closing </body> tag.

Step 2: Write the JavaScript Logic (scriptt.js)

1. Attach a submit event listener to the form element and call e.preventDefault() to stop default browser behavior.
2. Read and trim the values from the email and phone input fields.
3. Clear all previous error and success messages at the start of each submission.
4. Use validator.isEmail(email) to validate email; if invalid, display error message and set valid = false.
5. Use validator.isMobilePhone(phone, 'any') to validate phone; if invalid, display error message and set valid = false.
6. If all validations pass, display the success message "Form submitted successfully!".

Step 3: Apply CSS Styling (stylee.css)

1. Style the body using flexbox, centered content, steel-blue background (#4682b4), and full viewport height.
2. Style .form-container with a white background, 350px width, padding, box-shadow, and border-radius.
3. Style inputs to be 100% width with padding and rounded border.

4. Style `.error` class in red for validation error messages below inputs.
5. Style the submit button as full-width green with white text and darker green on hover.
6. Style `#successMessage` as centered, bold, green text below the form.

Step 4: Run the Application

1. Open the terminal and run: `start index.html` to launch the form in a browser.
2. Test invalid inputs — leave fields empty or enter incorrect formats and click Submit to verify error messages appear.
3. Test valid inputs — enter a properly formatted email and a valid phone number to confirm the success message.

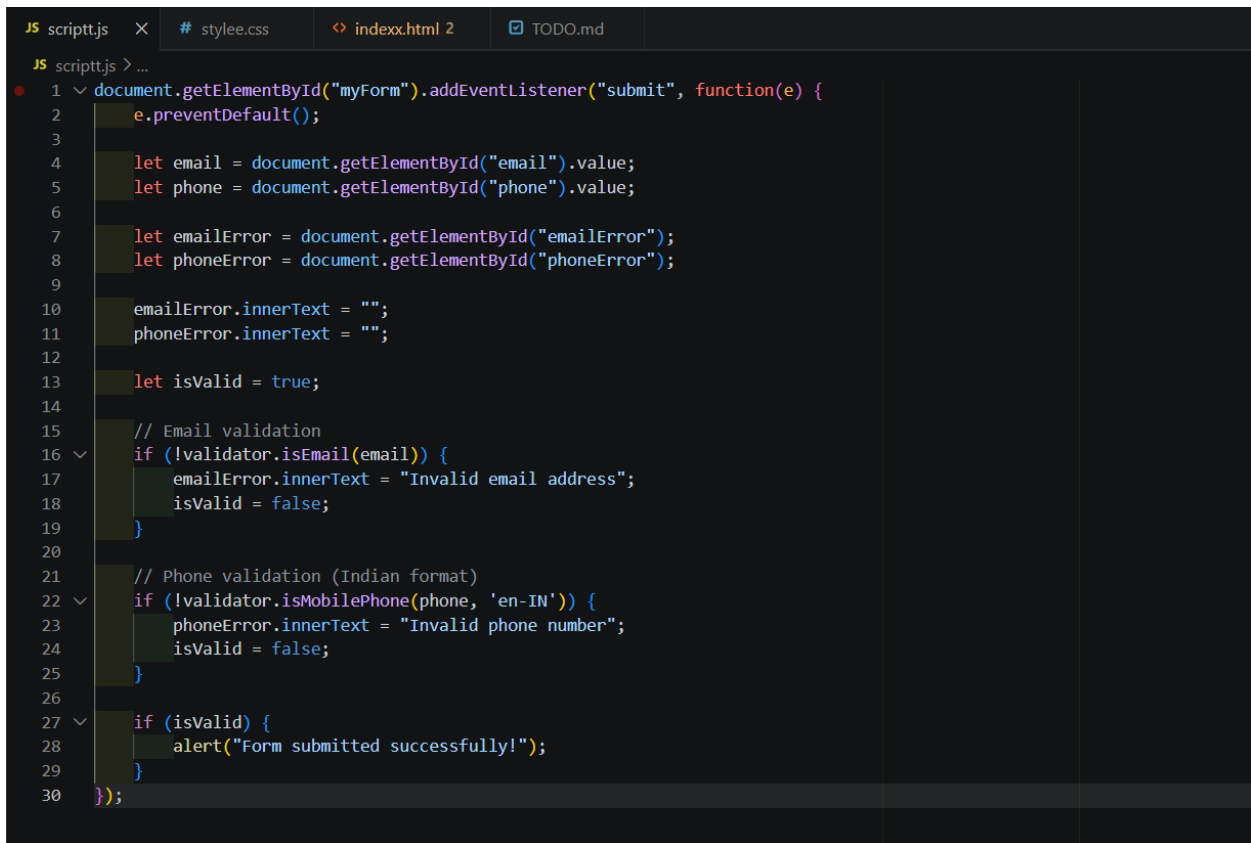
SCREENSHOTS

The following screenshots illustrate the code and working output of the experiment:

Source Code

```
JS scriptt.js # stylee.css indexx.html 2 x TODO.md
indexx.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Form Validation</title>
6
7   <!-- Link CSS -->
8   <link rel="stylesheet" href="stylee.css">
9
10  <!-- Validator.js CDN -->
11  <script src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.9.0/validator.min.js"></script>
12 </head>
13 <body>
14
15   <div class="container">
16     <h2>Register Form</h2>
17
18     <form id="myForm">
19       <label>Email:</label>
20       <input type="text" id="email">
21       <div id="emailError" class="error"></div>
22
23       <label>Phone Number:</label>
24       <input type="text" id="phone">
25       <div id="phoneError" class="error"></div>
26
27       <button type="submit">Submit</button>
28     </form>
29   </div>
30
31   <!-- Link JavaScript -->
32   <script src="scriptt.js"></script>
33
34 </body>
35 </html>
```

Fig 1: HTML Structure — indexx.html



The image shows a code editor with four tabs: 'scriptt.js', 'stylee.css', 'indexx.html 2', and 'TODO.md'. The 'scriptt.js' tab is active, displaying a JavaScript function that handles form submission. The function is attached to a submit event on an element with ID 'myForm'. It prevents the default form submission, retrieves the values of 'email' and 'phone' input fields, and checks for associated error messages. It then validates the email and phone numbers using a 'validator' object. If the email is invalid, it sets the error message to 'Invalid email address'. If the phone number is invalid (using 'en-IN' format), it sets the error message to 'Invalid phone number'. If both are valid, it shows an alert 'Form submitted successfully!'.

```
JS scriptt.js > ...
1 document.getElementById("myForm").addEventListener("submit", function(e) {
2     e.preventDefault();
3
4     let email = document.getElementById("email").value;
5     let phone = document.getElementById("phone").value;
6
7     let emailError = document.getElementById("emailError");
8     let phoneError = document.getElementById("phoneError");
9
10    emailError.innerText = "";
11    phoneError.innerText = "";
12
13    let isValid = true;
14
15    // Email validation
16    if (!validator.isEmail(email)) {
17        emailError.innerText = "Invalid email address";
18        isValid = false;
19    }
20
21    // Phone validation (Indian format)
22    if (!validator.isMobilePhone(phone, 'en-IN')) {
23        phoneError.innerText = "Invalid phone number";
24        isValid = false;
25    }
26
27    if (isValid) {
28        alert("Form submitted successfully!");
29    }
30 });
```

Fig 2: JavaScript Validation Logic — scriptt.js

```
JS script.js # stylee.css 5 X <> indexx.html 2 TODO.md
# stylee.css > button:hover
1 body {
2   font-family: Arial;
3   background: #f2f2f2;
4   display: flex;
5   justify-content: center;
6   align-items: center;
7   height: 100vh;
8 }
9
10 .container {
11   background: white;
12   padding: 20px;
13   border-radius: 10px;
14   width: 300px;
15   box-shadow: 0 0 10px gray;
16 }
17
18 h2 {
19   text-align: center;
20 }
21
22 input {
23   width: 100%;
24   padding: 8px;
25   margin: 10px 0;
26 }
27
28 .error {
29   color: red;
30   font-size: 12px;
31 }
32
33 button {
34   width: 100%;
35   padding: 10px;
36   background: green;
37   color: white;
```

Fig 3: CSS Styling — stylee.css

Program Output

Form Validation

file:///C:/Users/SWARNA/OneDrive/Documents/UID EXPERIMENT 9/indexx.html

Register Form

Email:

Phone Number:

Submit

